Question 1: What is a Vector Database (VectorDB) and how is it different from traditional databases?

Answer:

A **Vector Database (VectorDB)** is a specialized database designed to store, index, and search **high-dimensional vector embeddings** generated by machine learning models (e.g., text, image, audio embeddings). These vectors represent semantic meaning rather than exact values.

Difference between VectorDB and Traditional Databases

Aspect	Traditional Database	Vector Database
Data Type	Structured (rows, columns)	High-dimensional vectors
Query Type	Exact match, range queries	Similarity search (cosine, dot product, Euclidean)
Indexing	B-Tree, Hash Index	ANN (HNSW, IVF, FAISS)
Use Case	Transactions, CRUD operations	Semantic search, RAG, recommendations
Example	MySQL, PostgreSQL	ChromaDB, Pinecone, Weaviate

Conclusion:
Traditional databases answer **“Is this value equal?”**
Vector databases answer **“What is most similar in meaning?”**

Question 2: Explain the various types of VectorDBs available and their suitability for different use cases

Answer:

Vector databases can be classified based on **deployment model and architecture**.

1. In-Memory VectorDBs

- Example: FAISS
- Fast similarity search

- No persistence by default

Use Case:
Research experiments, offline ML pipelines

---

2. Persistent Local VectorDBs

- Example: ChromaDB
- Disk-based storage with metadata support

Use Case:
Local RAG systems, developer prototypes, small-scale apps

---

3. Cloud-Native VectorDBs

- Example: Pinecone, Weaviate
- Fully managed, scalable

Use Case:
Production AI systems, enterprise chatbots

---

4. Hybrid VectorDBs

- Combine vector + keyword search
- Example: Weaviate

Use Case:
Search engines requiring semantic + lexical filtering

Question 3: Why is Chroma DB important in AI/ML projects? Describe its key features

Answer:

ChromaDB is important because it provides a **simple, open-source, developer-friendly vector database** optimized for **LLM and RAG workflows**.

Key Features:

- Persistent vector storage
- Metadata filtering
- Embedding-agnostic
- Seamless integration with LangChain & LLMs
- Lightweight and local-first

Why it matters:

Without ChromaDB, developers must manually manage:

- Vector indexing
- Similarity computation
- Data persistence

ChromaDB abstracts all of this cleanly.

Question 4: What are the benefits of using Hugging Face Hub for generative AI tasks?

Answer:

Hugging Face Hub is a **centralized ecosystem** for AI models and datasets.

Benefits:

1. Thousands of pre-trained LLMs
2. Version-controlled model hosting
3. Easy integration via `transformers`

4. Open-source and community-driven
5. Supports fine-tuning and deployment

Bottom line:
It removes the need to train models from scratch.

Question 5: Describe the process and advantages of using pre-trained models from Hugging Face Hub

Answer:

Process:

1. Select model from Hugging Face Hub
2. Load using `transformers`
3. Tokenize input
4. Generate output or fine-tune

Advantages:

- Saves compute cost
- Faster development
- Proven architectures
- Supports domain adaptation

Question 6: Install and set up Chroma DB and insert sample vector data

Answer (Python Code):

```
# Install dependencies
```

```
pip install chromadb sentence-transformers

import chromadb
from sentence_transformers import SentenceTransformer

# Initialize Chroma client
client = chromadb.Client()

collection = client.create_collection(name="semantic_search")

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    "Artificial intelligence is transforming healthcare",
    "Machine learning improves medical diagnosis",
    "Vector databases enable semantic search"
]

embeddings = model.encode(documents).tolist()

collection.add(
    documents=documents,
    embeddings=embeddings,
    ids=["doc1", "doc2", "doc3"]
)

# Query
query = "AI in medicine"
query_embedding = model.encode([query]).tolist()

results = collection.query(
    query_embeddings=query_embedding,
    n_results=2
)

print(results)
```

Output (Sample):

```
{'documents': [['Artificial intelligence is transforming
healthcare',
```

```
                'Machine learning improves medical diagnosis']]}
```

Question 7: Download and fine-tune a Hugging Face model for text generation

Answer (Python Code):

```
from transformers import AutoModelForCausalLM, AutoTokenizer,
Trainer, TrainingArguments

model_name = "gpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=2
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=None  # Placeholder for dataset
)

print("Model ready for fine-tuning")
```

Output:

```
Model ready for fine-tuning
```

Question 8: Create a custom LLM using Ollama and Llama2

Answer:

Steps:

1. Install Ollama
2. Pull Llama2
3. Run locally

Commands:

```
ollama pull llama2
ollama run llama2
```

Prompt Example:

```
Explain vector databases
```

Output (Sample):

```
Vector databases store embeddings for similarity search...
```

Question 9: Implement a basic RAG system using Ollama with Llama3

Answer (Python Code):

```
from sentence_transformers import SentenceTransformer
import chromadb
import subprocess

model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.Client()
collection = client.create_collection("rag")

docs = ["LLMs use context", "RAG improves factual accuracy"]
embeddings = model.encode(docs).tolist()

collection.add(documents=docs, embeddings=embeddings, ids=["1",
"2"])

query = "How does RAG help?"
```

```
q_emb = model.encode([query]).tolist()

retrieved = collection.query(query_embeddings=q_emb, n_results=1)

context = retrieved["documents"][0][0]

prompt = f"Context: {context}\nQuestion: {query}"

subprocess.run(["ollama", "run", "llama3", prompt])
```

Question 10: Health-tech chatbot using Hugging Face + VectorDB + Ollama

Answer:

Proposed Architecture:

1. **Hugging Face Model**
    - Bio-clinical embeddings (e.g., BioBERT)
2. **VectorDB (ChromaDB)**
    - Store medical research embeddings
3. **Ollama (Llama3)**
    - Generate grounded responses

Flow:

User Query → Embedding → Vector Search → Context Retrieval → LLM Generation

Advantages:

- Accurate retrieval
- Reduced hallucinations
- Local, secure inference

- Scalable architecture

Conclusion:

This system ensures **fact-grounded medical responses**, which is mandatory in health-tech.